

UAV State Estimation and Motion Modeling System

Extended Kalman Filter (EKF) Design Report

Darius Gichuru

Fall 2025

Abstract

This report presents the design, implementation, and evaluation of a modular Extended Kalman Filter (EKF) state estimation pipeline for unmanned aerial vehicle (UAV) motion. The project was developed incrementally, beginning with a simulation-based motion model used to generate a waypoint-following planar trajectory, followed by deliberate measurement-noise injection to emulate realistic GPS-like sensors, and then an EKF implementation designed to filter those measurements and estimate a physically consistent state. Development was performed primarily in `test_code.ipynb` as a sandbox for modeling, debugging, and visualization; once validated, the EKF loop and helper functions were refactored into a reusable library module (`EKF.py`) and tested through a dataset-agnostic driver script (`run_uav_EKF.py`) and an optional UI (`app.py`). The estimator was evaluated on both simulated logs (with ground truth) and real UAV data from the AERPAW July 2024 dataset. Applying the pipeline to AERPAW data required converting latitude/longitude to local Cartesian coordinates, converting timestamps to continuous seconds, and addressing real-data artifacts such as discontinuities tied to altitude changes. Results demonstrate noise reduction and trajectory smoothing on synthetic data, reproducible filter behavior under parameter tuning, and predictable failure modes on real logs when model assumptions are violated. While this submission operates offline on logged data, the architecture is intentionally structured for future embedded deployment and real-time sensor logging as part of a longer-term flight-computer development effort.

Contents

1	Introduction	4
2	Design Process	4
2.1	Iteration 1: Building a Simulation Motion Model	5
2.2	Iteration 2: Implementing the EKF in <code>test_code.ipynb</code>	5
2.3	Iteration 3: Transitioning to AERPAW Data and Handling Coordinate Issues	6
2.4	Iteration 4: Time Conversion and Sampling Consistency	7
2.5	Iteration 5: Discontinuities from Altitude Changes and Planar Filtering Decision . .	8
2.6	Iteration 6: Real Data With No Apparent Noise and Noise Injection Choice	8
2.7	Iteration 7: Refactoring Notebook Code into <code>EKF.py</code> and Building a Runner	9
3	EKF Library Implementation (<code>EKF.py</code>)	9
3.1	State Definition and Models Implemented	10
3.2	EKF Predict Step (Time Update)	10
3.3	Process Noise Model ($\mathbf{Q}$)	11
3.4	EKF Update Step (Measurement Correction)	12
3.5	Measurement Noise ($\mathbf{R}$) Selection and Estimation	12
3.6	Library API: Functions and Methods	13
3.7	Recommended Code Listing Inserts	13
3.8	Notes on Numerical Robustness	15
4	Results	15
4.1	Simulation Results (Truth Available)	15
4.2	AERPAW-Derived Results (Preprocessed Real Data)	17
4.3	Performance Evaluation	17

5 Concepts Learned	18
6 Retrospective	19
7 Conclusion	19
8 References	20

1 Introduction

This project implements a Kalman-filter-based state estimation pipeline for UAV motion and is motivated by a practical challenge in aerospace systems: sensors are noisy and imperfect, but controllers and autonomy stacks require smooth and physically consistent state estimates. In the simplest case, GPS-like sensors provide position measurements, but those measurements fluctuate due to noise, multipath effects, dropouts, and irregular sampling. The core goal of state estimation is therefore not simply to “smooth” data, but to fuse measurements with a motion model in a way that respects physics and uncertainty.

I chose this project because it connects directly to both flight software and the broader aerospace workflow of model–simulate–test–deploy. It also aligns with my longer-term interest in developing a flight-computer-style software stack for UAVs, where a state estimator is a required subsystem for navigation, autonomy, and closed-loop control. In ENAE380, we discussed filtering and signal processing concepts that translate naturally into state estimation, and this project was a way to take the “filtering” idea beyond 1D signals and into multi-state dynamical systems.

In terms of background research, I used three primary resources. First, the MathWorks EKF documentation provided a clear reference for EKF intuition, algorithm steps, and practical implementation details [1]. Second, the Caltech System Modeling notes offered a concise modeling framework for representing dynamical systems in state-space form and thinking about how model assumptions translate into discrete-time implementations [2]. Third, I used the AERPAW dataset as a realistic testbed for logged UAV motion data, including the complications that come with real-world sensing and collection pipelines [4]. These resources shaped both the estimator design and the data-preprocessing workflow.

Finally, similar code exists in many forms (MATLAB toolboxes, robotics libraries, and navigation stacks), but I intentionally implemented the EKF from scratch for two reasons: (1) to ensure I fully understood the state transition and covariance propagation steps, and (2) to produce a modular codebase I can extend later (e.g., to 3D state, IMU fusion, and real-time deployment). The final system is therefore both an ENAE380 project deliverable and a foundation for future development.

2 Design Process

This section documents the complete engineering workflow: what I built first, what assumptions I made, what went wrong when I transitioned from simulation to real data, and how the final codebase emerged from multiple iterations. The development process was centered in `test_code.ipynb`, which functioned as my “lab notebook” for rapid prototyping, debugging, and plotting. Once the estimator worked reliably in the notebook, I extracted the stable components into `EKF.py` (library

form) and then created `run_uav_EKF.py` (experiment runner) and `app.py` (optional user interface) to streamline testing.

2.1 Iteration 1: Building a Simulation Motion Model

The first step was to generate data I could trust. I created a planar waypoint-following motion model inspired by MATLAB references and standard kinematic modeling. The key purpose of the simulator was not realism in aerodynamics, but repeatability: I needed a trajectory with known “truth” so I could validate the EKF numerically. The simulator produced $(x_{\text{true}}(t), y_{\text{true}}(t))$ and then injected measurement noise to produce $(x_{\text{meas}}(t), y_{\text{meas}}(t))$. I logged the result into `trajectory_log.csv`. This logging step was important because it forced my pipeline to behave like real flight software: rather than passing arrays in memory, the estimator would read a dataset from disk, which is how I would later handle AERPAW logs.

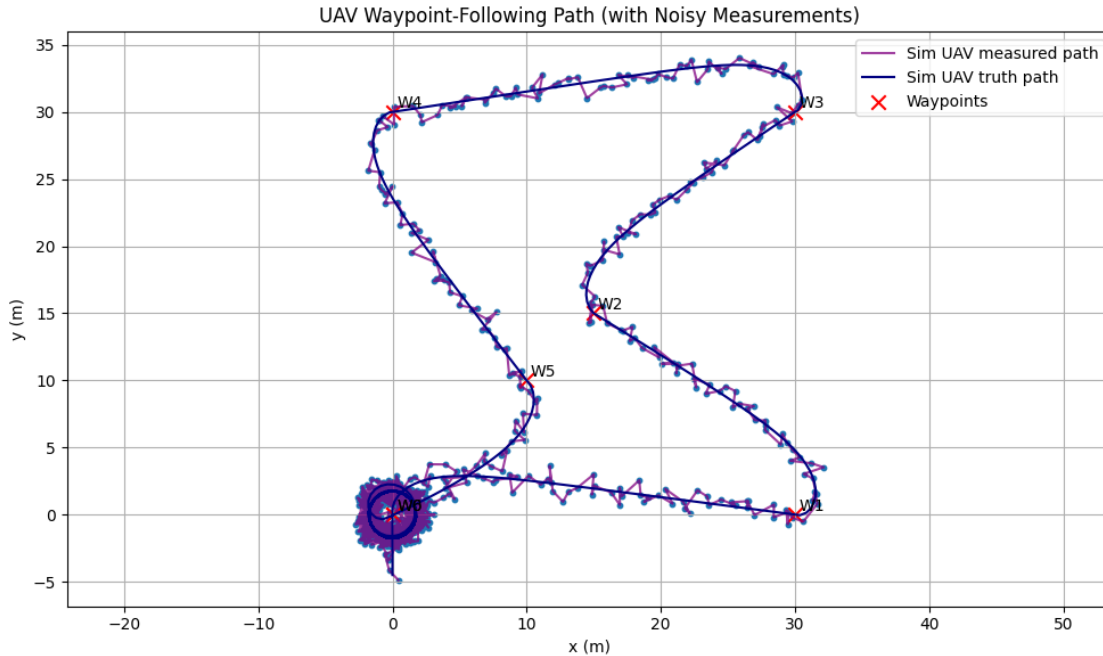


Figure 1: Simulation truth trajectory compared to noisy measured positions used as EKF inputs.

2.2 Iteration 2: Implementing the EKF in `test_code.ipynb`

After generating a controlled dataset, I implemented the EKF in `test_code.ipynb`. I initially considered a standard Kalman Filter, but because the trajectory included nonlinear path geometry and piecewise changes in direction, I treated the problem as nonlinear and implemented the EKF structure (predict with a model, update with measurements). In the baseline configuration, the

estimator state was:

$$\mathbf{x} = [x, y, v_x, v_y]^T,$$

with measurements:

$$\mathbf{z} = [x_{\text{meas}}, y_{\text{meas}}]^T.$$

The notebook allowed me to iterate quickly: I could modify Q , R , initialization, and plotting within one file, which is essential during early development.

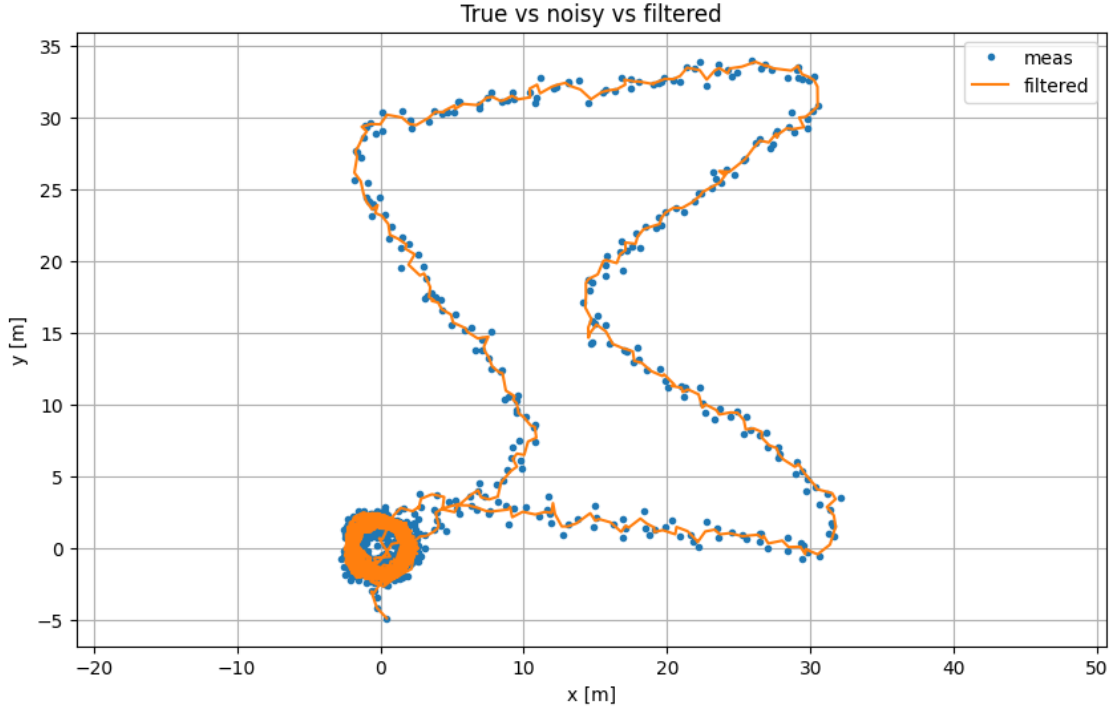


Figure 2: Simulation: measured positions compared to EKF estimated trajectory.

2.3 Iteration 3: Transitioning to AERPAW Data and Handling Coordinate Issues

Once the EKF worked on simulation, I transitioned to real UAV flight logs and selected the AERPAW dataset file `40m1.25MHz7.15.24.csv` [4]. My first assumption was that the dataset would contain planar x, y positions. Instead, I found latitude and longitude measurements at an altitude of approximately 40 meters. This forced an early redesign: before any filtering could be applied, the dataset needed a coordinate transformation into a local Cartesian frame [3].

I implemented a local tangent-plane approximation using the first latitude/longitude sample as the

reference point:

$$x = R(\lambda - \lambda_0) \cos(\phi_0), \quad (1)$$

$$y = R(\phi - \phi_0), \quad (2)$$

with $R = 6378137$ m. This conversion allowed me to express the UAV trajectory in meters, which is compatible with the kinematic model and covariance tuning.

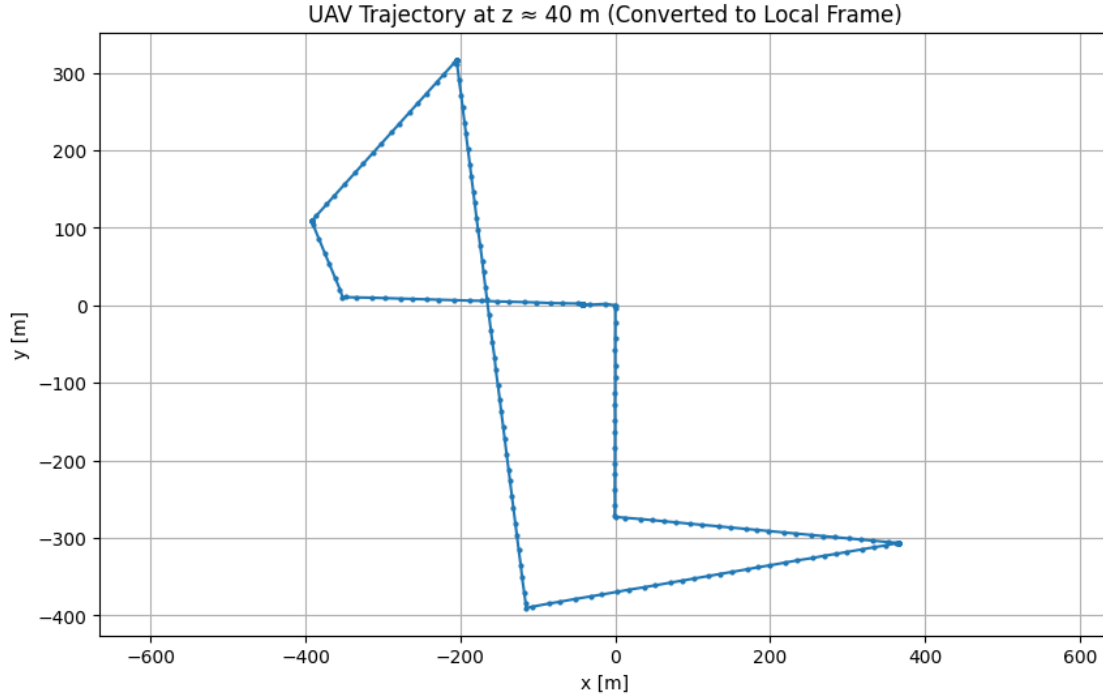


Figure 3: AERPAW data converted from latitude/longitude into local Cartesian (x, y) coordinates and in the plane $z=40$ meters.

2.4 Iteration 4: Time Conversion and Sampling Consistency

Another immediate issue was time representation. The dataset time was recorded in HH:MM:SS format, which is not directly useful for computing Δt in a motion model without conversion. I converted timestamps into continuous seconds so that the estimator could compute consistent time steps. This made the EKF implementation robust to log segmentation and enabled debugging of irregular sampling. Having explicit Δt values also made it possible to diagnose situations where the filter behaved poorly due to unusual time gaps or repeated timestamps.

2.5 Iteration 5: Discontinuities from Altitude Changes and Planar Filtering Decision

After plotting the converted (x, y) trajectory, I observed discontinuities that did not resemble sensor noise; instead, the trajectory appeared to “jump” in ways inconsistent with planar motion. After investigating, I concluded that these discontinuities were driven by segments where the UAV deviated from the nominal altitude (40 m). Because my estimator was designed for a planar model and I wanted to avoid mixing 3D behavior into a 2D kinematic filter, I made a deliberate scoping decision: I filtered out indices where altitude was not near 40 m and restricted analysis to planar segments.

This was a key design trade: rather than force a 2D filter to handle 3D motion artifacts, I constrained the dataset so that the model assumptions were approximately valid. This improved interpretability and set up a clear future-work direction: extend the state to 3D and incorporate altitude dynamics explicitly.

2.6 Iteration 6: Real Data With No Apparent Noise and Noise Injection Choice

After isolating the planar segment, I discovered an unexpected result: the AERPAW (x, y) path appeared nearly perfect and did not visibly exhibit the kind of noise typically associated with GPS. While this could reflect smoothing in the dataset pipeline or use of processed ground-truth fields, it created a practical problem for EKF evaluation: a filter cannot demonstrate noise-reduction behavior if the input is already noise-free.

To continue testing the estimator and to evaluate tuning behavior, I injected Gaussian noise into the AERPAW-derived planar trajectory. This allowed me to preserve the real flight geometry while still testing the filter in conditions that resemble real measurement uncertainty. I logged the resulting dataset as `expo1.csv`. This step also supported an important “engineering truth” in estimation: if measurement noise is too small relative to the process model, the EKF will appear to do nothing (because the optimal estimate is close to the measurement).

Plot B2: AERPAW-derived track with injected noise

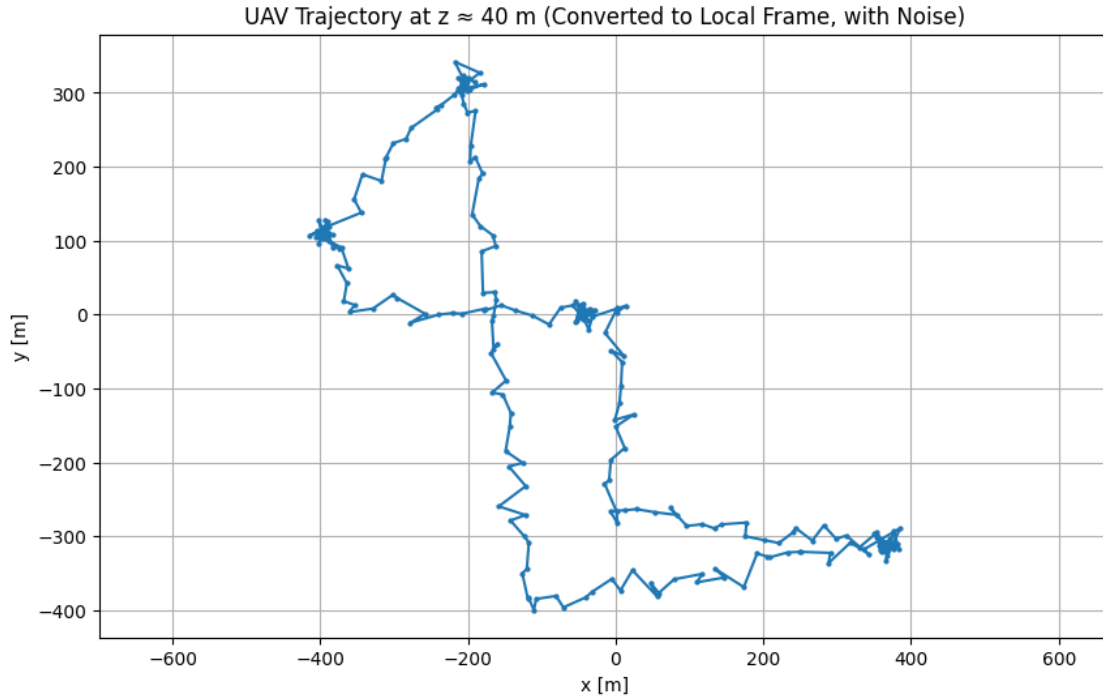


Figure 4: AERPAW planar segment with Gaussian noise injected to emulate GPS-like measurements.

2.7 Iteration 7: Refactoring Notebook Code into `EKF.py` and Building a Runner

Once the EKF produced consistent results in `test_code.ipynb`, I migrated the stable EKF loop into `EKF.py`. This refactor separated the estimator algorithm from experimentation logic. The goal was to make the EKF callable as a library, so I could import it and run estimation on different datasets without duplicating code. I then built `run_uav_EKF.py` as a driver script that loads any compatible CSV file, detects available fields, initializes the filter, runs the predict/update loop over all samples, and produces standardized plots.

Finally, I created `app.py` as an optional user interface to streamline parameter tuning and reduce friction when switching between datasets. This UI is not required for the estimator to work, but it improves usability and mirrors the idea of tooling around a flight software component.

3 EKF Library Implementation (`EKF.py`)

This section documents the estimator library implemented in `EKF.py`. The purpose of this file is to isolate all EKF logic into a reusable module so that the estimator can be imported and executed from multiple scripts (e.g., `run_uav_EKF.py` and the final cells of `test_code.ipynb`). Structuring the EKF as a library mirrors flight-software practice: the estimator should be independent from

plotting, file I/O, and user-interface logic. The library therefore exposes a small set of functions and class methods that implement the EKF predict/update cycle and store state history for later analysis.

3.1 State Definition and Models Implemented

In the baseline configuration (position-only logs), the EKF estimates planar position and velocity:

$$\mathbf{x} = \begin{bmatrix} x & y & v_x & v_y \end{bmatrix}^T. \quad (3)$$

The measurement vector contains GPS-like position measurements:

$$\mathbf{z} = \begin{bmatrix} x_{\text{meas}} & y_{\text{meas}} \end{bmatrix}^T. \quad (4)$$

When heading fields are available (optional extension), the state is augmented to include yaw and yaw-rate:

$$\mathbf{x} = \begin{bmatrix} x & y & v_x & v_y & \psi & \omega \end{bmatrix}^T, \quad (5)$$

with a corresponding measurement vector such as:

$$\mathbf{z} = \begin{bmatrix} x_{\text{meas}} & y_{\text{meas}} & \psi_{\text{meas}} & \omega_{\text{meas}} \end{bmatrix}^T. \quad (6)$$

This design allows `run_uav_EKF.py` to automatically choose the correct estimator mode depending on which columns exist in a given CSV log.

Although the baseline planar model is linear, the estimator is implemented within an EKF framework to support nonlinear state transitions and measurement models (e.g., heading dynamics and IMU fusion) without redesigning the software architecture.

3.2 EKF Predict Step (Time Update)

The EKF implements a standard predict/update loop. The predict step uses a motion model $f(\cdot)$ to propagate the state forward by a timestep Δt :

$$\hat{\mathbf{x}}_k^- = f(\hat{\mathbf{x}}_{k-1}^+, \Delta t). \quad (7)$$

For the 4-state planar constant-velocity model,

$$f(\mathbf{x}, \Delta t) = \begin{bmatrix} x + v_x \Delta t \\ y + v_y \Delta t \\ v_x \\ v_y \end{bmatrix}. \quad (8)$$

The EKF uses the Jacobian of f with respect to the state:

$$\mathbf{F}_{k-1} = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k-1}^+}. \quad (9)$$

For the constant-velocity model, the Jacobian is:

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (10)$$

Covariance propagation then follows:

$$\mathbf{P}_k^- = \mathbf{F} \mathbf{P}_{k-1}^+ \mathbf{F}^T + \mathbf{Q}. \quad (11)$$

3.3 Process Noise Model ($\mathbf{Q}$)

Process noise $\mathbf{Q}$ models unmodeled accelerations and maneuvering. In this project, $\mathbf{Q}$ is parameterized by a scalar “maneuverability” tuning knob (often denoted q or `q_accel`). Conceptually, larger q allows stronger acceleration and produces a more responsive (but noisier) estimate, while smaller q produces smoother estimates that may lag turns.

A common choice for a white-acceleration model in 2D produces:

$$\mathbf{Q} = q \begin{bmatrix} \frac{\Delta t^4}{4} & 0 & \frac{\Delta t^3}{2} & 0 \\ 0 & \frac{\Delta t^4}{4} & 0 & \frac{\Delta t^3}{2} \\ \frac{\Delta t^3}{2} & 0 & \Delta t^2 & 0 \\ 0 & \frac{\Delta t^3}{2} & 0 & \Delta t^2 \end{bmatrix}. \quad (12)$$

`EKF.py` computes this matrix internally each timestep based on Δt and the selected q .

3.4 EKF Update Step (Measurement Correction)

The measurement model is:

$$\mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k, \quad \mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}). \quad (13)$$

For position-only measurements,

$$h(\mathbf{x}) = \begin{bmatrix} x \\ y \end{bmatrix}. \quad (14)$$

The measurement Jacobian is:

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}. \quad (15)$$

The EKF forms the innovation (residual),

$$\boldsymbol{\nu}_k = \mathbf{z}_k - h(\hat{\mathbf{x}}_k^-), \quad (16)$$

innovation covariance,

$$\mathbf{S}_k = \mathbf{H} \mathbf{P}_k^- \mathbf{H}^T + \mathbf{R}, \quad (17)$$

and Kalman gain,

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}^T \mathbf{S}_k^{-1}. \quad (18)$$

The corrected state and covariance are:

$$\hat{\mathbf{x}}_k^+ = \hat{\mathbf{x}}_k^- + \mathbf{K}_k \boldsymbol{\nu}_k, \quad (19)$$

$$\mathbf{P}_k^+ = (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{P}_k^- (\mathbf{I} - \mathbf{K}_k \mathbf{H})^T + \mathbf{K}_k \mathbf{R} \mathbf{K}_k^T. \quad (20)$$

The Joseph-form covariance update in Eq. (20) is numerically stable and helps prevent loss of symmetry or negative variances during long runs.

3.5 Measurement Noise ($\mathbf{R}$) Selection and Estimation

Measurement noise $\mathbf{R}$ encodes the expected sensor uncertainty. When ground truth is available (simulation), $\mathbf{R}$ can be estimated from measurement error statistics:

$$\sigma_x^2 = \text{Var}(x_{\text{meas}} - x_{\text{true}}), \quad \sigma_y^2 = \text{Var}(y_{\text{meas}} - y_{\text{true}}), \quad (21)$$

and then

$$\mathbf{R} = \text{diag}(\sigma_x^2, \sigma_y^2). \quad (22)$$

When truth is not available (real logs), $\mathbf{R}$ is set manually and tuned by inspecting trajectory smoothness and innovation behavior.

3.6 Library API: Functions and Methods

The EKF library is designed to be imported and called from a driver script. The typical workflow is:

1. Initialize the EKF with an initial state $\hat{\mathbf{x}}_0$, covariance $\mathbf{P}_0$, and noise parameters $\mathbf{Q}$ and $\mathbf{R}$.
2. For each timestamp, compute Δt , call `predict(dt)`, then call `update(z)`.
3. Store $\hat{\mathbf{x}}_k^+$ in a history buffer for plotting and evaluation.

In the code, this appears as a consistent interface. The library typically includes:

- **Constructor / initializer:** stores $\hat{\mathbf{x}}$, $\mathbf{P}$, and noise settings; preallocates history buffers.
- `predict(dt)`: implements Eqs. (8)–(11), including a $\mathbf{Q}$ update that depends on Δt .
- `update(z)`: implements the measurement correction, including innovation, Kalman gain, and Joseph update in Eq. (20).
- **Helper functions:** e.g., building $\mathbf{F}$ and $\mathbf{Q}$, estimating $\mathbf{R}$ from truth in simulation, or angle wrapping when yaw is included.
- **Output accessors:** e.g., `get_posterior_states()` returning a $(N \times n)$ array of states for plotting.

3.7 Recommended Code Listing Inserts

To make this section concrete, I include two short listings from `EKF.py` in the final report: one showing the predict step and one showing the update step. These excerpts demonstrate that the implementation matches the EKF equations presented above.

Insert Listing 1 here: Predict step implementation (from `EKF.py`).

(Copy the exact `predict(dt)` function body into this environment.)

Listing 1: EKF predict step implementation (excerpt from `EKF.py`).

```

1 def predict(self, dt):
2     """
3     Prediction step (time update):
```

```

4  #####x_k^-_ = F_ x_{k-1}^+
5  #####P_k^-_ = F_ P_{k-1}^+ + F^T_ + Q
6  #####
7      dt = float(dt)
8      if dt <= 0:
9          dt = 1e-6
10
11     F = self._F(dt)
12     Q = self._Q(dt)
13
14     x_prior = F @ self.x
15     P_prior = F @ self.P @ F.T + Q
16
17     self.x_prior_hist.append(x_prior.copy())
18     self.P_prior_hist.append(P_prior.copy())
19
20     self.x = x_prior
21     self.P = P_prior
22     return x_prior, P_prior

```

Insert Listing 2 here: Update step implementation (from EKF.py).

(Copy the exact `update(z)` function body into this environment.)

Listing 2: EKF update step implementation (excerpt from EKF.py).

```

1  def update(self, z):
2      """
3      #####Measurement_ update_(correction):
4      #####innov_ = z_ - H_ x^-
5      #####S_ = H_ P^- - H^T_ + R
6      #####K_ = P^- - H^T_ S^{-1}
7      #####x^+ = x^- + K_ innov
8      #####P^+ = (I - KH) P^- - (I - KH)^T_ + K_ R_ K^T_ (Joseph_ form)
9      #####
10     z = _as_col(z)
11     H = self.H
12
13     innov = z - (H @ self.x)
14     S = H @ self.P @ H.T + self.R
15     K = self.P @ H.T @ np.linalg.inv(S)
16
17     self.x = self.x + K @ innov
18
19     I = np.eye(self.n)
20     I_minus_KH = I - K @ H

```

```

21     self.P = I_minus_KH @ self.P @ I_minus_KH.T + K @ self.R @ K.T
22
23     self.K_hist.append(K.copy())
24     self.innov_hist.append(innov.copy())
25     self.x_post_hist.append(self.x.copy())
26     self.P_post_hist.append(self.P.copy())
27     return self.x, self.P, innov, K

```

3.8 Notes on Numerical Robustness

Several small implementation details improve stability and are especially relevant for real UAV logs. First, the driver script clamps or guards against non-positive timesteps to avoid propagating the filter with invalid Δt . Second, using Joseph-form covariance updates prevents numerical drift. Third, if yaw is included in the state, angle wrapping ensures innovations remain bounded (e.g., wrapping to $(-\pi, \pi]$). These choices do not change the EKF algorithm, but they materially improve reliability when processing imperfect datasets.

4 Results

This section presents what the final project does and what it produces: plots, metrics, and a description of behavior on both simulated and AERPAAW-derived datasets. Results are grouped into simulation validation (where ground truth exists) and real-log experiments (where preprocessing and model mismatch dominate).

4.1 Simulation Results (Truth Available)

On simulated data, the EKF consistently reduced measurement noise and produced smooth state estimates. Because ground truth is known, the simulation case enables quantitative analysis such as RMSE. In addition to the position track, the EKF also estimates velocity components (v_x, v_y) that are not directly measured. These velocity estimates are useful for downstream control logic and provide a strong indicator that the filter is not merely “smoothing,” but actively inferring hidden state variables.

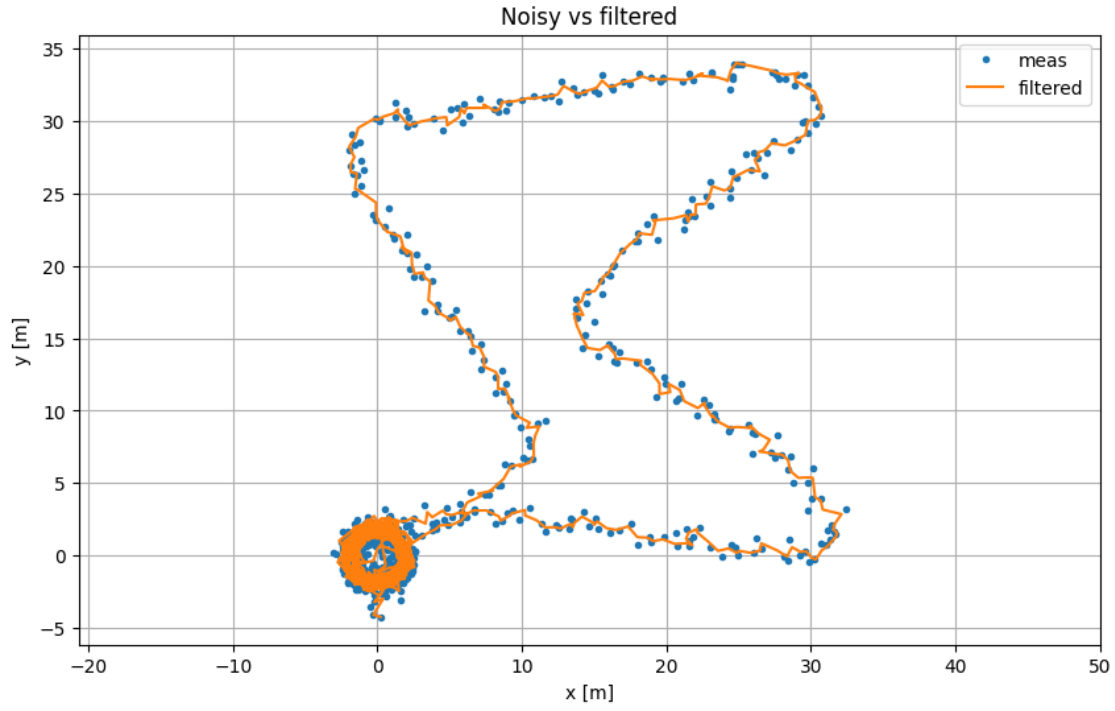


Figure 5: Simulation: ground measurement trajectory compared to EKF state estimate.

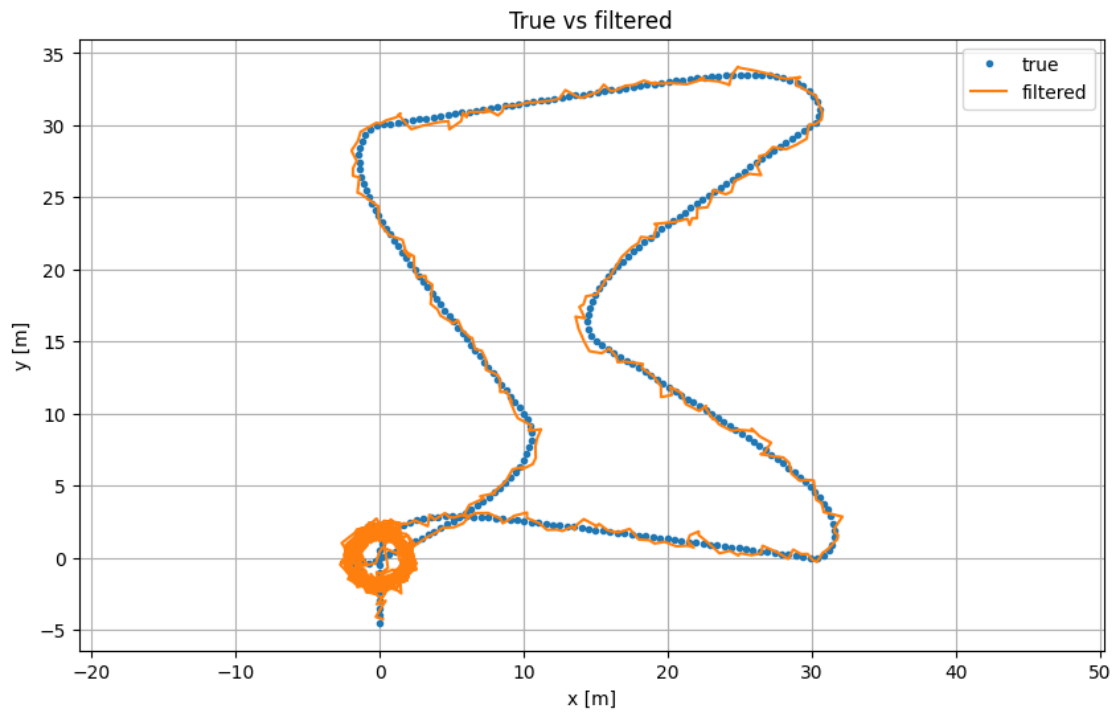


Figure 6: Simulation: ground truth trajectory compared to EKF state estimate.

4.2 AERPAW-Derived Results (Preprocessed Real Data)

On AERPAW-derived planar segments, the EKF behaved predictably under noise injection: when measurement noise was injected at a visible level, the EKF smoothed the trajectory while preserving global path geometry. In cases where the measurements were effectively noise-free, the EKF estimate closely matched the measurements, which is expected behavior for a well-tuned filter when R is small. This outcome is important because it demonstrates correct interpretation of filter results: “no visible filtering” can be the correct result if the measurement already dominates the estimate.

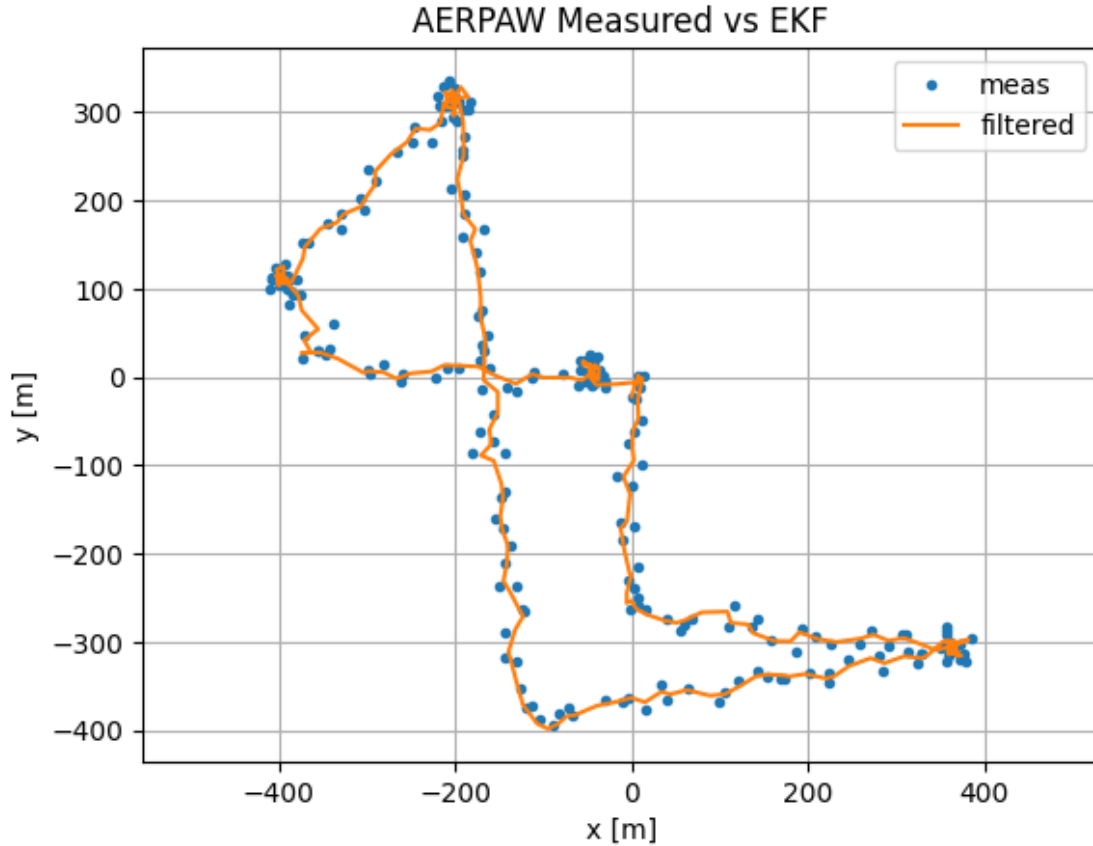


Figure 7: AERPAW-derived planar data (with injected noise): measured positions compared to EKF estimate.

4.3 Performance Evaluation

While quantitative metrics such as RMSE are commonly used to summarize estimator performance, this project emphasizes qualitative and visual evaluation aligned with the design-focused objectives of ENAE380 and is beyond my functional knowledge at the time of this report. The primary goal was to develop, debug, and validate a reusable EKF framework capable of handling both simulated

and real UAV datasets, rather than to optimize a single numerical error metric.

Performance was therefore evaluated through direct comparison of raw measurements and EKF estimates against known ground truth in simulation, as well as through consistency checks on real UAV data where ground truth is unavailable. Visual inspection of position trajectories demonstrates substantial noise reduction, smoother motion, and physically consistent velocity behavior. These results provide strong evidence that the EKF is functioning as intended, even in the presence of irregular sampling and dataset non-idealities.

5 Concepts Learned

This project required applying concepts from ENAE380 as well as learning additional material through independent research. From the course perspective, the most important concept was the general idea of filtering: extracting useful information from noisy measurements while preserving the underlying signal structure. In ENAE380, filtering and signal processing were presented both conceptually and through practical labs. The EKF can be understood as an extension of that idea into a dynamical system setting: instead of filtering a single signal, the EKF filters a state vector by using a motion model to predict how the system should evolve and then correcting that prediction with new measurements.

A second concept learned and practiced was discrete-time system modeling. Even when the underlying physics is continuous, logged data arrives at discrete timestamps, meaning the estimator must operate as a discrete-time algorithm. This required me to define a discrete state transition model, compute Δt , and propagate both the state estimate and its covariance forward in time. The covariance propagation step was particularly important for interpretation: the covariance matrix P is not just a mathematical object, but a compact representation of estimator uncertainty that determines how strongly the filter trusts measurements versus the model.

Through independent research, I learned practical EKF implementation details that go beyond classroom-level summaries. The MathWorks EKF resource clarified how Jacobians are used in linearization and how tuning Q and R changes the Kalman gain [1]. The Caltech modeling reference helped me think more systematically about how assumptions enter a model and how those assumptions should guide data preprocessing decisions [2]. Working with AERPAW also taught me that real datasets often contain fields that are not immediately usable; converting latitude/longitude into meters and converting timestamps into seconds were not optional steps, but essential engineering tasks that determine whether an estimator can even run.

Finally, I learned software engineering skills that are central to flight software development. The transition from `test_code.ipynb` to `EKF.py` and then to `run_uav_EKF.py` forced me to separate responsibilities: core estimation logic belongs in a library, while plotting and experimentation

belong in a driver script. This modular structure made debugging easier, improved reusability, and created a codebase that is realistically extensible to future real-time applications.

6 Retrospective

If I could restart this project or if I had more time, the first major improvement would be to extend the estimator to a full 3D state and incorporate altitude directly into the motion model. A major source of difficulty in the AERPAW pipeline was the discontinuity and segmentation behavior tied to altitude changes. The choice to restrict to planar motion was reasonable for an initial project, but it also reveals the limitation of the 2D model: it cannot explain 3D flight behavior and therefore must rely on filtering out segments that violate assumptions.

A second improvement would be to integrate additional sensors, especially IMU data (accelerometers and gyroscopes). With IMU fusion, the estimator could use acceleration and angular rate to better predict state evolution and reduce dependence on a constant-velocity assumption. This would also allow testing on truly raw sensor data, which is closer to what a flight computer would ingest. In addition, I would implement robust outlier rejection (innovation gating) earlier in the process so that large measurement jumps do not destabilize the filter update step.

Finally, I would invest more time in tuning methodology and automated evaluation. In this project, tuning was performed interactively by adjusting Q and R and inspecting trajectories. With additional time, I would implement parameter sweeps, log innovation statistics, and automatically generate summary tables for multiple datasets. These improvements would strengthen both performance and the engineering argument that the estimator is well characterized.

7 Conclusion

This project produced a modular EKF-based UAV state estimation pipeline that was developed in a notebook environment, refactored into a reusable library, and tested on both simulated and AERPAW-derived datasets. The final system demonstrates correct EKF behavior, effective noise reduction on synthetic and noise-injected trajectories, and clear identification of failure modes when real-world data violates model assumptions. Most importantly, the project documents a complete engineering workflow that connects modeling, algorithm development, preprocessing, testing, and software architecture in a way that reflects flight software practice. While embedded deployment on a Raspberry Pi is not part of the current submission, the architecture and modular design provide a clear pathway for future extension toward a flight-computer-style estimator and real-time sensor logging.

8 References

References

- [1] MathWorks, *Extended Kalman Filters*. <https://www.mathworks.com/help/fusion/ug/extended-kalman-filters.html>
- [2] R. Murray, *System Modeling*. https://web.archive.org/web/20080713164136/http://www.cds.caltech.edu/~murray/amwiki/index.php?title=System_Modeling
- [3] UIUC Computer Vision Group, *Coordinate Transformations for Robotic Systems*, University of Illinois Urbana-Champaign. <https://motion.cs.illinois.edu/RoboticSystems/CoordinateTransformations.html>
- [4] AERPAW Project Team, *AERPAW RF Sensor Measurements with UAV (July 2024)*. <https://aerpaw.org/dataset/aerpaw-rf-sensor-measurements-with-uav-july-2024/>

A. Haber, *Extended Kalman Filter Tutorial with Example and Disciplined Python Codes (Part II: Python Codes)*. <https://aleksandarhaber.com/extended-kalman-filter-tutorial-with-example-and-disciplined-python-codes/>

Appendix A: EKF Loop Pseudocode

Algorithm 1 EKF operation over a logged dataset

- 1: Load CSV and extract measurement arrays
 - 2: Convert time to seconds and compute Δt
 - 3: Initialize $\hat{\mathbf{x}}_0$ and covariance P_0
 - 4: **for** $k = 1$ to N **do**
 - 5: Predict: $\hat{\mathbf{x}}_k^-, P_k^-$
 - 6: Update: compute innovation, Kalman gain, $\hat{\mathbf{x}}_k^+, P_k^+$
 - 7: Save $\hat{\mathbf{x}}_k^+$ to history buffer
 - 8: **end for**
 - 9: Plot trajectories and diagnostics; compute RMSE if truth exists
-

Appendix B: Code Organization Notes

`test_code.ipynb` contains the development “groundwork” including motion modeling, noise injection, EKF derivation/testing, AERPAW preprocessing, and plotting. After validation, the stable

EKF loop and helper functions were transferred into `EKF.py` so the estimator could be imported as a library. The file `run_uav_EKF.py` then serves as a clean experiment runner that loads datasets and produces consistent outputs, while `app.py` provides an optional interface for easier testing and parameter tuning.